

PREMIUM TRAINING GUIDE

VictoriaMetrics

as an LLM Observability Backend

Concrete setup, MetricsQL, dashboards, alerts and sovereign architecture

Samuel Desseaux

Erythix
VictoriaMetrics Training Partner - France / Benelux /
Germany

Table of Contents

Module 1 : Why VictoriaMetrics for LLM workloads	3
Module 2 : Complete architecture VM + OTel	5
<i>Diagram 1 : Full architecture overview</i>	6
Module 3 : Single Node vs Cluster - choosing your deployment mode	8
<i>Diagram 2 : Single vs Cluster comparison</i>	9
Module 4 : Step-by-step installation and configuration	11
Module 5 : LLM metrics taxonomy	16
<i>Diagram 3 : Complete LLM metrics taxonomy</i>	17
Module 6 : Python instrumentation - pushing metrics	20
Module 7 : MetricsQL - essential queries for LLM observability	25
<i>Diagram 4 : MetricsQL vs PromQL - key features</i>	26
Module 8 : Grafana dashboards - structure and panels	31
<i>Diagram 5 : Recommended Grafana LLM dashboard layout</i>	32
Module 9 : VMAAlert - production alert rules	36
Module 10 : Recording rules - pre-aggregating cost metrics	40
Module 11 : Optimization, retention and tuning	43
Module 12 : Security, air-gap and compliance	46
Module 13 : Practical labs	48
Module 14 : References and resources	51

1. Why VictoriaMetrics for LLM Workloads

1.1 The specific challenge of LLM metrics

LLM workloads in production generate an atypical metrics profile that pushes classic monitoring backends to their limits:

LLM Characteristic	Impact on the metrics backend
High cardinality	model * provider * tenant * pipeline = thousands of timeseries. Prometheus saturates quickly.
Token histograms	Wide distributions (10 to 100k tokens/request). Very spread-out buckets.
USD cost metrics	Financial metrics requiring float precision. Aggregations by accounting period.
Long retention	Cost audits, contractual SLAs: 1-3 year retention is often required.
Ingestion bursts	A completion batch can generate 10x the normal write rate in seconds.
RAG quality metrics	Floats, distributions, scores: write-intensive, frequently read for alerting.

1.2 VictoriaMetrics vs Prometheus - head-to-head

Criteria	Prometheus	VictoriaMetrics
Cardinality	Degrades above 500k active series	Stable up to 10M+ active series
Compression	~4:1 ratio	~10:1 ratio (20:1 with long retention)
Write throughput	~200k samples/s per instance	>1M samples/s on a standard instance
Long retention	Degrades after 90 days	Performant over 2+ years without manual compaction
Query language	PromQL	MetricsQL (superset: anomaly_score, median_over_time...)
Native OTLP	Via Collector only	Direct OTLP endpoint in Single mode (port 4318)
Deployment	Multiple recommended components	Single binary in Single mode
RAM footprint	4 GB RAM / 500k series	1 GB RAM / 500k series (4x less)

Criteria	Prometheus	VictoriaMetrics
Migration	N/A	100% Prometheus-compatible API: zero dashboard migration

1.3 Learning objectives

By the end of this guide, you will be able to:

- Deploy VictoriaMetrics in Single mode in 15 minutes for an LLM development environment
- Choose between Single and Cluster mode based on your actual LLM workload
- Configure the OTel Collector -> VictoriaMetrics pipeline for LLM metrics collection
- Instrument a Python/FastAPI application to emit the right LLM metrics
- Write MetricsQL queries for 8 essential use cases (cost, quality, latency, GPU)
- Build a Grafana LLM dashboard with 4 functional rows and real-time alerts
- Configure VMAalert with production-grade alert rules for LLM degradations
- Optimize retention, storage and recording rules to minimize infra costs

2. Complete Architecture VM + OTel for LLM

2.1 Overview

The architecture below covers the full metrics collection path from LLM sources to dashboards and alerts. It is deployable in full air-gap mode.

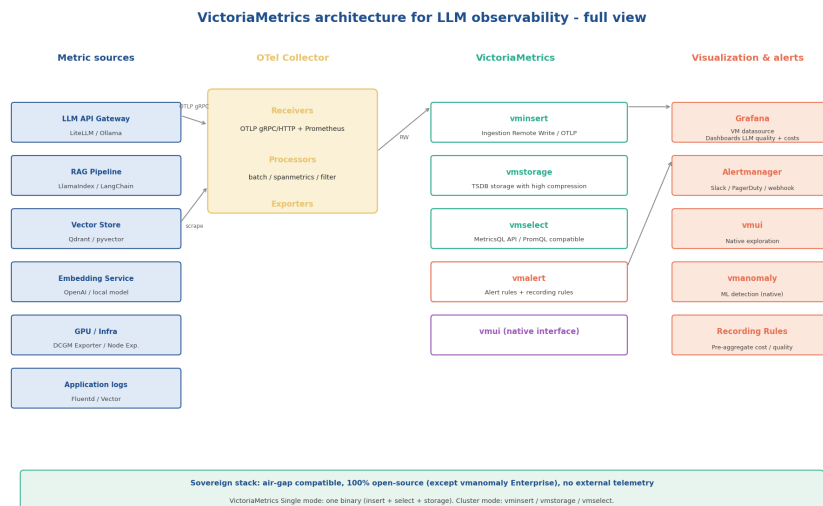


Diagram 1 : Complete VictoriaMetrics architecture for LLM observability - air-gap compatible

Key architectural points

All sources emit to the OTel Collector via OTLP (gRPC port 4317 or HTTP port 4318)
 The Collector filters, enriches (k8s labels), derives metrics from traces (spanmetrics), and exports
 VictoriaMetrics receives via Remote Write. In Single mode, it also accepts OTLP directly.
 VMAAlert evaluates MetricsQL rules and sends alerts to Alertmanager
 Grafana uses VictoriaMetrics as a datasource (Prometheus-compatible, zero reconfiguration)
 vmanomaly (Enterprise) adds ML-based anomaly detection on LLM metrics

2.2 Components and roles

Component	Port(s)	Role for LLM observability
OTel Collector	4317 (gRPC), 4318 (HTTP)	Normalises and routes all metrics. spanmetrics derives metrics from RAG traces.
VictoriaMetrics Single	8428	TSDB storage, MetricsQL API, Remote Write, direct OTLP, built-in vmui

Component	Port(s)	Role for LLM observability
vmagent	8429	Distributed Prometheus scraping. Ideal for DCGM Exporter, vLLM /metrics, Ollama
vmalert	8880	Evaluation of MetricsQL alert rules. Talks to Alertmanager.
Alertmanager	9093	Alert routing: Slack, PagerDuty, email, webhook. Deduplication and silencing.
Grafana	3000	Dashboards. Uses the VictoriaMetrics plugin or standard Prometheus datasource.
vmanomaly (Enterprise)	9090	ML anomaly detection (ARIMA, Prophet) on LLM metrics. Optional.

3. Single Node vs Cluster - Choosing Your Mode

3.1 Comparison diagram

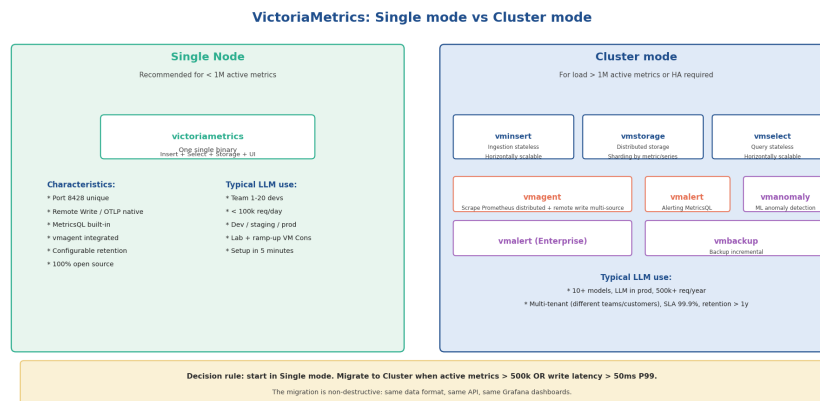


Diagram 2 : Single mode (one binary) vs Cluster mode (3 separate components) - LLM decision criteria

3.2 Decision criteria for an LLM context

Criteria	Single mode	Cluster mode
Active metrics	< 1M (recommended < 500k)	> 500k, theoretically unlimited
LLM requests/s	< 500 rps	> 500 rps or intensive multi-tenant
High availability	Not native (external solutions)	Native: replicas per component
LLM models in prod	1 to 10 models	> 10 models or > 50 tenants
Deployment	1 container / 1 binary	3 Kubernetes deployments minimum
Retention	Configurable, identical	Configurable per tier (hot/warm/cold)
Ops complexity	Very low	Moderate to high
RAM required	1-4 GB depending on load	4 GB+ per component
License	Open source (Apache 2.0)	Open source (vminsert/vmstorage/vmselect)

Migration rule: Single to Cluster

Migrate when: active metrics > 500k OR P99 write latency > 50ms OR HA SLA required.
Migration is non-destructive: same data format, same API, same Grafana dashboards.

Procedure: vmstorage snapshot -> restore on new cluster. Zero downtime possible with vmagent.

4. Installation and Configuration Step by Step

4.1 Docker Compose - full stack in 5 minutes

Development or lightweight production stack: VictoriaMetrics Single + OTel Collector + Grafana + VMAAlert.

```
# docker-compose.yml
version: '3.8'
services:

  victoriametrics:
    image: victoriametrics/victoria-metrics:v1.99.0
    command:
      - '--storageDataPath=/var/lib/victoria-metrics-data'
      - '--retentionPeriod=12'          # 12 months retention
      - '--httpListenAddr=:8428'
      - '--search.maxQueryLen=16384' # Long MetricsQL queries
    ports: ['8428:8428']
    volumes:
      - vm-data:/var/lib/victoria-metrics-data

  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.99.0
    command: ['--config=/etc/otel-collector-config.yaml']
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - '4317:4317' # OTLP gRPC
      - '4318:4318' # OTLP HTTP
    depends_on: [victoriametrics]

  vmagent:
    image: victoriametrics/vmagent:v1.99.0
    command:
      - '--promscrape.config=/etc/prometheus.yml'
      - '--remoteWrite.url=http://victoriametrics:8428/api/v1/write'
    volumes:
      - ./prometheus.yml:/etc/prometheus.yml

  vmaalert:
    image: victoriametrics/vmaalert:v1.99.0
    command:
      - '--datasource.url=http://victoriametrics:8428'
      - '--remoteWrite.url=http://victoriametrics:8428/api/v1/write'
      - '--notifier.url=http://alertmanager:9093'
      - '--rule=/etc/alerts/*.yaml'
    volumes:
      - ./alerts:/etc/alerts

  alertmanager:
    image: prom/alertmanager:v0.27.0
```

```

volumes: ['./alertmanager.yml:/etc/alertmanager/alertmanager.yml']
ports: ['9093:9093']

grafana:
  image: grafana/grafana:10.4.0
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_INSTALL_PLUGINS=victoriametrics-metrics-datasource
  ports: ['3000:3000']

volumes:
  vm-data: {}

```

4.2 OTEL Collector configuration for LLM metrics

```

# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
  prometheus:
    config:
      scrape_configs:
        - job_name: vllm
          scrape_interval: 15s
          static_configs: [{ targets: ['vllm-service:8000'] }]
        - job_name: ollama
          scrape_interval: 15s
          static_configs: [{ targets: ['ollama:11434'] }]

processors:
  batch: { timeout: 5s, send_batch_size: 512 }
  k8sattributes:
    extract:
      metadata: [k8s.pod.name, k8s.namespace.name, k8s.deployment.name]
  spanmetrics:
    metrics_exporter: prometheusremotewrite
    latency_histogram_buckets: [50, 100, 200, 500, 1000, 2000, 5000, 10000]
    dimensions:
      - name: gen_ai.request.model
      - name: gen_ai.system
      - name: rag.index.id
      - name: rag.pipeline.id
      - name: rag.tenant.id

exporters:
  prometheusremotewrite:
    endpoint: http://victoriametrics:8428/api/v1/write
    timeout: 30s
    retry_on_failure: { enabled: true, initial_interval: 5s }

service:
  pipelines:
    metrics:

```

```
    receivers: [otlp, prometheus]
    processors: [k8sattributes, batch]
    exporters: [prometheusremotewrite]
traces/metrics:
    receivers: [otlp]
    processors: [spanmetrics, batch]
    exporters: [prometheusremotewrite]
```

4.3 vmagent - scraping DCGM and LLM endpoints

```
# prometheus.yml (used by vmagent)
global:
  scrape_interval: 15s
  external_labels:
    cluster: 'prod-llm'
    environment: 'production'

scrape_configs:
  # DCGM Exporter - NVIDIA GPU metrics
  - job_name: 'dcm-exporter'
    scrape_interval: 5s
    kubernetes_sd_configs: [{ role: pod }]
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: dcm-exporter
        action: keep

  # vLLM inference server
  - job_name: 'vllm'
    static_configs:
      - targets: ['vllm-llama:8000', 'vllm-mistral:8001']

  # Ollama (local models)
  - job_name: 'ollama'
    static_configs: [{ targets: ['ollama:11434'] }]

  # LiteLLM proxy
  - job_name: 'litellm'
    static_configs: [{ targets: ['litellm-proxy:4000'] }]
```

5. LLM Metrics Taxonomy

The taxonomy below covers the four dimensions of LLM observability: response quality, performance and latency, costs and tokens, and GPU/CPU infrastructure.

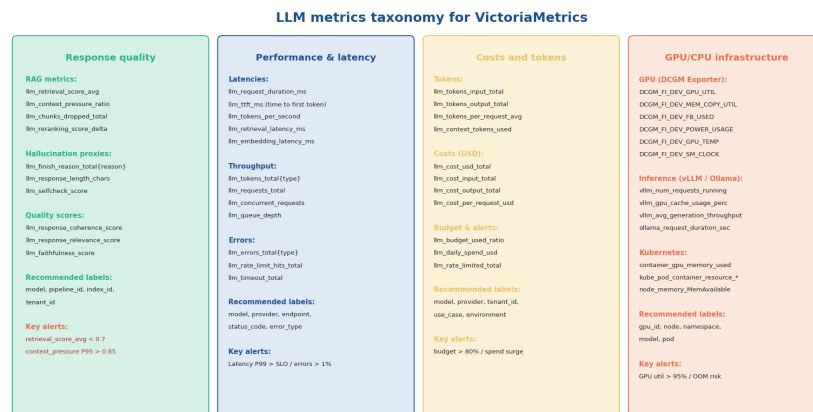


Diagram 3 : Complete LLM metrics taxonomy for VictoriaMetrics - 4 dimensions, recommended labels, alert thresholds

5.1 Recommended naming conventions

- Prefix `llm_` for all LLM-specific metrics
- Prefix `rag_` for retrieval-specific metrics
- Use DCGM standard names for GPU metrics (`DCGM_FI_*`)
- Standard suffixes: `_total` (counter), `_seconds/_ms` (latency histogram), `_ratio` (0-1 gauge), `_bytes` (gauge)

Metric	Prometheus Type	Cardinality	Essential Labels
llm_request_duration_ms	Histogram	Medium	model, provider, pipeline_id, status
llm_tokens_total{type}	Counter	Low	model, provider, type=input output
llm_cost_usd_total	Counter	Low	model, provider, tenant_id, use_case
llm_errors_total	Counter	Medium	model, provider, error_type, status_code
llm_ttf_ms	Histogram	Medium	model, provider, stream=true false
rag_retrieval_score_avg	Gauge	Low	index_id, pipeline_id, strategy
rag_context_pressure_ratio	Histogram	Low	pipeline_id, model, tenant_id
rag_chunks_dropped_total	Counter	Low	pipeline_id, reason
DCGM_FI_DEV_GPU_UTIL	Gauge	Low	gpu, UUID, pod, namespace, node

Metric	Prometheus Type	Cardinality	Essential Labels
vllm:num_requests_running	Gauge	Low	model_name, instance

Cardinality warning: tenant_id label

If you have more than 1000 tenants, the tenant_id label can cause a cardinality explosion. Solution: use a tenant_tier label (free/pro/enterprise) for high-frequency metrics, and keep tenant_id only for cost metrics (llm_cost_usd_total) which have low write frequency.

6. Python Instrumentation - Pushing Metrics

6.1 Setup: two complementary approaches

- Prometheus client library: /metrics endpoint exposed + scraping by vmagent. Simple, battle-tested.
- OTLP via OTel SDK: metrics AND traces in the same pipeline. Recommended for new applications.

```
# Option 1: prometheus_client
pip install prometheus-client

# Option 2: OTel SDK (unified metrics + traces)
pip install opentelemetry-sdk opentelemetry-exporter-otlp-proto-grpc
```

6.2 Prometheus client: core LLM metrics

```
from prometheus_client import Counter, Histogram, Gauge, start_http_server
import time

LLM_REQUESTS = Counter(
    'llm_requests_total', 'Total LLM requests',
    ['model', 'provider', 'pipeline_id', 'status']
)

LLM_DURATION = Histogram(
    'llm_request_duration_ms', 'LLM request duration in ms',
    ['model', 'provider', 'pipeline_id'],
    buckets=[50, 100, 200, 500, 1000, 2000, 5000, 10000, 30000]
)

LLM_TOKENS = Counter(
    'llm_tokens_total', 'Total tokens processed',
    ['model', 'provider', 'type'] # type: input|output
)

LLM_COST = Counter(
    'llm_cost_usd_total', 'Total cost in USD',
    ['model', 'provider', 'tenant_id', 'use_case']
)

LLM_TTFT = Histogram(
    'llm_ttft_ms', 'Time to first token in ms',
    ['model', 'provider'],
    buckets=[50, 100, 200, 500, 1000, 2000, 5000]
)

RAG_RETRIEVAL_SCORE = Gauge(
    'rag_retrieval_score_avg', 'Average retrieval similarity score',
    ['index_id', 'pipeline_id', 'strategy']
)

RAG_CONTEXT_PRESSURE = Histogram(
    'rag_context_pressure_ratio', 'Context window pressure ratio (0-1)',
    ['pipeline_id', 'model'],
    buckets=[0.1, 0.3, 0.5, 0.7, 0.8, 0.85, 0.9, 0.95, 1.0]
)
```

```
# Expose /metrics on port 9090
start_http_server(9090)
```

6.3 Instrumentation decorator - automatic cost tracking

```
import functools

TOKEN_PRICES = {
    'gpt-4o': {'input': 5.0e-6, 'output': 15.0e-6},
    'gpt-4o-mini': {'input': 0.15e-6, 'output': 0.6e-6},
    'claude-3-5-sonnet': {'input': 3.0e-6, 'output': 15.0e-6},
    'mistral-large': {'input': 4.0e-6, 'output': 12.0e-6},
}

def track_llm_call(model, provider, pipeline_id, tenant_id='default',
use_case='chat'):
    def decorator(func):
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            labels = {'model': model, 'provider': provider, 'pipeline_id':
pipeline_id}
            t0 = time.monotonic()
            status = 'success'
            try:
                result = await func(*args, **kwargs)
                if hasattr(result, 'usage'):
                    in_tok = result.usage.prompt_tokens
                    out_tok = result.usage.completion_tokens
                    LLM_TOKENS.labels(model=model, provider=provider,
type='input').inc(in_tok)
                    LLM_TOKENS.labels(model=model, provider=provider,
type='output').inc(out_tok)
                    if model in TOKEN_PRICES:
                        p = TOKEN_PRICES[model]
                        cost = in_tok * p['input'] + out_tok * p['output']
                        LLM_COST.labels(model=model, provider=provider,
tenant_id=tenant_id,
use_case=use_case).inc(cost)
                return result
            except Exception as e:
                status = type(e).__name__
                raise
            finally:
                LLM_DURATION.labels(**labels).observe((time.monotonic()-
t0)*1000)
                LLM_REQUESTS.labels(**labels, status=status).inc()
            return wrapper
        return decorator

@track_llm_call(model='gpt-4o', provider='openai', pipeline_id='support-rag',
tenant_id='acme')
async def call_llm(messages: list) -> dict:
    return await openai_client.chat.completions.create(model='gpt-4o',
messages=messages)
```

7. MetricsQL - Essential Queries

7.1 MetricsQL features for LLM observability

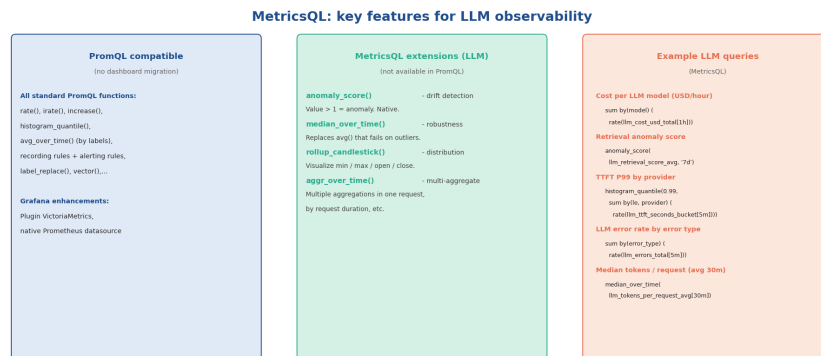


Diagram 4 : MetricsQL vs PromQL - key features for LLM workloads

7.2 Cost and token queries

Hourly LLM cost by model (USD)

```
sum by (model, provider) (increase(llm_cost_usd_total[1h]))
```

Daily cost projection (extrapolated from last 30 minutes)

```
sum by (model) (rate(llm_cost_usd_total[30m])) * 86400
```

Top 10 tenants by cost (last 24h)

```
topk(10, sum by (tenant_id) (increase(llm_cost_usd_total[24h])))
```

Input/output token ratio (detect outlier requests)

```
sum(rate(llm_tokens_total{type='output'}[5m])) /
sum(rate(llm_tokens_total{type='input'}[5m]))
```

7.3 RAG quality queries

Retrieval score anomaly - index drift detection

```
# anomaly_score: value > 1 = significant deviation vs 7-day baseline
anomaly_score(
  avg by (index_id) (rag_retrieval_score_avg),
  '7d'
)
```


Context pressure P95 by pipeline

```
histogram_quantile(0.95,
  sum by (le, pipeline_id) (
    rate(rag_context_pressure_ratio_bucket[10m])
  )
)
```

Requests with dropped chunks (%)

```
sum by (pipeline_id) (rate(rag_chunks_dropped_total[5m]))
/
sum by (pipeline_id) (rate(llm_requests_total[5m]))
* 100
```

7.4 Performance and latency queries

End-to-end P99 latency by model

```
histogram_quantile(0.99,
  sum by (le, model, provider) (
    rate(llm_request_duration_ms_bucket[5m])
  )
)
```

TTFT P95 - time to first token (critical for streaming)

```
histogram_quantile(0.95,
  sum by (le, model) (rate(llm_ttft_ms_bucket[5m]))
)
```

GPU energy efficiency - tokens per watt

```
sum by (model) (rate(llm_tokens_total{type='output'}[5m]))
/
avg by (model) (DCGM_FI_DEV_POWER_USAGE)
```

8. Grafana Dashboards - Structure and Panels

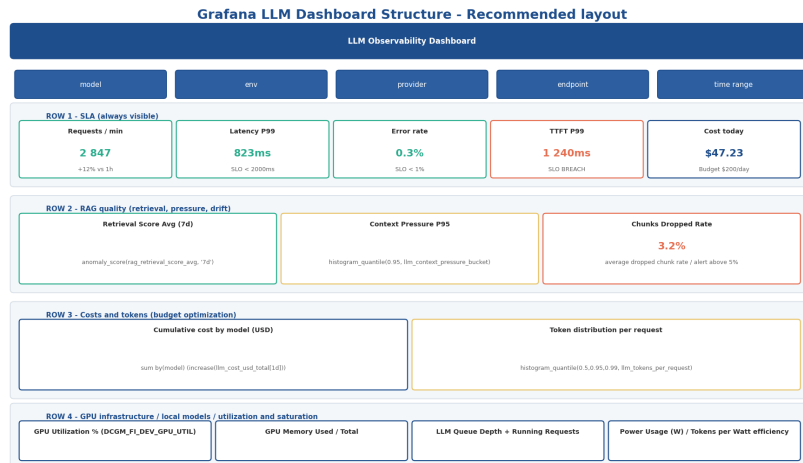


Diagram 5 : Grafana LLM dashboard layout - 4 functional rows with MetricsQL queries

8.1 Datasource configuration

```
# Auto-provision datasource (grafana/provisioning/datasources/vm.yaml)
apiVersion: 1
datasources:
  - name: VictoriaMetrics
    type: victoriametrics-metrics-datasource
    url: http://victoriametrics:8428
    access: proxy
    isDefault: true
    jsonData:
      timeInterval: '15s'
```

8.2 Template variables

```
# Variable 'model': list of active models
label_values(llm_requests_total, model)

# Variable 'provider'
label_values(llm_requests_total{model=~'$model'}, provider)

# Variable 'pipeline_id'
label_values(rag_retrieval_score_avg, pipeline_id)
```

8.3 Recommended panel types per metric

Metric	Panel type	Key options
Requests/min (SLA)	Stat + sparkline	Thresholds: green >0, orange if drift >20%
P99 latency	Stat + threshold	Absolute thresholds per contractual SLO
Error rate	Stat + threshold	Red >1%, orange >0.5%
Retrieval score 7 days	Time series	anomaly_score line + 0.70 threshold
Context pressure P95	Time series + threshold	Red zone above 0.85
Cumulative cost by model	Stacked area time series	Color per model, USD unit
Token distribution	Heatmap or Histogram	Visualise outliers (abnormally long requests)
GPU utilisation	Gauge + threshold	Orange threshold at 80%, red at 95%
vLLM queue depth	Time series	Alert if >10 requests waiting
Anomaly score	Time series	Threshold at 1.0 (significant deviation)

9. VMAAlert - Production Alert Rules

9.1 Complete production ruleset

```
# alerts/llm-quality.yml
groups:
  - name: llm_quality
    interval: 1m
    rules:

      # RAG index drift
      - alert: RAGIndexDrift
        expr: |
          avg by (index_id, pipeline_id) (
            avg_over_time(rag_retrieval_score_avg[1h])
          ) < 0.70
        for: 15m
        labels: { severity: warning, team: ai-platform }
        annotations:
          summary: 'RAG index drift on {{ $labels.index_id }}'
          description: |
            Mean retrieval score = {{ $value | printf "%.2f" }} < threshold
            0.70
            for 15m on pipeline {{ $labels.pipeline_id }}.
            Action: check for document updates without re-indexing.
            runbook_url: 'https://wiki.internal/runbooks/rag-index-drift'

      # Context pressure critical
      - alert: RAGContextPressureHigh
        expr: |
          histogram_quantile(0.95,
            sum by (le, pipeline_id) (
              rate(rag_context_pressure_ratio_bucket[10m])
            )
          ) > 0.85
        for: 5m
        labels: { severity: warning, team: ai-platform }
        annotations:
          summary: 'High context pressure P95: {{ $labels.pipeline_id }}'
          description: 'P95 = {{ $value | printf "%.2f" }}. Reduce system
            prompt or increase context window.'

      # High LLM error rate
      - alert: LLMHighErrorRate
        expr: |
          sum by (model, provider) (rate(llm_errors_total[5m]))
          /
          sum by (model, provider) (rate(llm_requests_total[5m]))
          > 0.01
        for: 3m
        labels: { severity: critical, team: ai-platform }
        annotations:
```

```

        summary: 'Error rate > 1% on {{ $labels.model }} / {{
$labels.provider }}'
        description: 'Current rate: {{ $value | printf "%.1f" }}%. Check
provider availability.'

# Latency SLO breach
- alert: LLMLatencySLOBreach
  expr: |
    histogram_quantile(0.99,
      sum by (le, model, pipeline_id) (
        rate(llm_request_duration_ms_bucket[5m])
      )
    ) > 5000
  for: 5m
  labels: { severity: critical, team: ai-platform }
  annotations:
    summary: 'P99 latency SLO breach on {{ $labels.model }}'
    description: 'P99 = {{ $value | printf "%.0f" }}ms > SLO 5000ms.
Pipeline {{ $labels.pipeline_id }}'

# GPU memory saturation
- alert: GPUMemorySaturation
  expr: DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL * 100 > 90
  for: 2m
  labels: { severity: warning, team: mlops }
  annotations:
    summary: 'GPU {{ $labels.gpu }} memory saturated ({{ $value | printf
("%.0f" }}%)'
    description: 'OOM risk. Reduce batch size or model context window.'

# Daily LLM budget
- alert: LLMDailyBudgetWarning
  expr: |
    sum by (tenant_id) (
      increase(llm_cost_usd_total[24h])
    ) > 80
  labels: { severity: warning, team: billing }
  annotations:
    summary: 'LLM budget at 80% for tenant {{ $labels.tenant_id }}'
    description: '24h cost = ${{ $value | printf "%.2f" }}. Threshold:
$100/day.'

```

9.2 Alertmanager routing by team

```

# alertmanager.yml
route:
  group_by: ['alertname', 'model', 'pipeline_id']
  group_wait: 30s
  repeat_interval: 12h
  receiver: 'default'
  routes:
    - matchers: [severity='critical']
      receiver: pagerduty
    - matchers: [team='ai-platform']
      receiver: slack-ai-platform
    - matchers: [team='billing']

```

```
receiver: slack-billing
```

```
receivers:
```

```
- name: 'default'
```

```
  slack_configs:
```

```
    - channel: '#llm-ops-alerts'
```

```
      title: '{{ .CommonAnnotations.summary }}'
```

```
      text: '{{ .CommonAnnotations.description }}'
```

```
- name: 'pagerduty'
```

```
  pagerduty_configs:
```

```
    - routing_key: 'YOUR_PAGERDUTY_KEY'
```

10. Recording Rules - Pre-aggregating Cost Metrics

Cost metrics are queried very frequently in dashboards. Recalculating `increase(llm_cost_usd_total[24h])` on every refresh is CPU-intensive. Recording rules pre-compute these aggregations in the background, yielding a 10-100x performance improvement.

```
# alerts/recording-rules.yml
groups:
  - name: llm_recording_rules
    interval: 1m
    rules:

      - record: llm:cost_usd:rate1h
        expr: |
          sum by (model, provider, tenant_id) (
            increase(llm_cost_usd_total[1h])
          )

      - record: llm:cost_usd:rate24h
        expr: |
          sum by (model, provider, tenant_id) (
            increase(llm_cost_usd_total[24h])
          )

      - record: llm:error_rate:ratio5m
        expr: |
          sum by (model, provider) (rate(llm_errors_total[5m]))
          /
          sum by (model, provider) (rate(llm_requests_total[5m]))

      - record: llm:request_duration_ms:p99_5m
        expr: |
          histogram_quantile(0.99,
            sum by (le, model, pipeline_id) (
              rate(llm_request_duration_ms_bucket[5m])
            )
          )

      - record: llm:ttft_ms:p95_5m
        expr: |
          histogram_quantile(0.95,
            sum by (le, model) (rate(llm_ttft_ms_bucket[5m]))
          )

      - record: rag:retrieval_score:avg30m
        expr: |
          avg by (index_id, pipeline_id) (
            avg_over_time(rag_retrieval_score_avg[30m])
          )
```

```
- record: rag:context_pressure:p95_10m
  expr: |
    histogram_quantile(0.95,
      sum by (le, pipeline_id) (
        rate(rag_context_pressure_ratio_bucket[10m])
      )
    )

- record: gpu:memory_utilization:ratio
  expr: DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL
```

Using recording rules in alerts and dashboards

Once defined, use recording rules in alerts and dashboards instead of the full query:

Instead of: `histogram_quantile(0.99, sum by(le,model)(rate(llm_request_duration_ms_bucket[5m])))`

Use: `llm:request_duration_ms:p99_5m`

The query is 50x faster and the result is identical.

11. Optimization, Retention and Tuning

11.1 Retention policy by metric type

Metric type	Recommended retention	Justification	VM flag
USD cost metrics	2-3 years	Accounting audit, contractual SLAs	--retentionPeriod=24m
RAG quality scores	6-12 months	Long-term drift detection	--retentionPeriod=12m
Latency and throughput	3-6 months	Performance trending	--retentionPeriod=6m
GPU metrics	1-3 months	Capacity planning	--retentionPeriod=3m
OTel traces (Jaeger)	7-30 days	Debug, no long-term audit	Jaeger TTL config

11.2 Cardinality optimisation

- Never label by trace_id, request_id or session_id - infinite cardinality.
- Restrict tenant_id to cost metrics only (not for latency or token metrics).
- Group similar models under a model_family label if more than 20 active versions.
- Use adapted histogram buckets for tokens: [10, 100, 500, 1000, 4096, 16384, 65536].

```
# Monitor cardinality in VictoriaMetrics
# Via API: http://victoriametrics:8428/api/v1/status/tsdb

# Top 10 metrics by cardinality (MetricsQL)
topk(10, count by (__name__) ({__name__!=""}))
```

11.3 VictoriaMetrics tuning flags for LLM production

```
victoriametrics \
  --storageDataPath=/var/lib/vm-data \
  --retentionPeriod=12 \
  --httpListenAddr=:8428 \
  --maxInsertRequestSize=32MB \           # Batch LLM writes
  --search.maxQueryLen=16384 \           # Long MetricsQL queries
  --search.maxSamplesPerQuery=1e9 \      # Large time windows for cost audit
  --memory.allowedPercent=60 \          # Leave 40% for the kernel
  --loggerLevel=INFO \
  --loggerFormat=json
```

12. Security, Air-Gap and Compliance

12.1 Authentication and access control

- VictoriaMetrics has no native authentication. Use vmauth (open-source) or an Nginx reverse proxy.
- vmauth supports role-based routing: read-only for Grafana, write-only for OTel Collector, admin for ops.
- Kubernetes NetworkPolicy: restrict access to VM ports to authorised components only.

```
# vmauth config
users:
  - username: grafana
    password: '${GRAFANA_PASSWORD}'
    url_prefix:
      - 'http://victoriametrics:8428/api/v1/query'
      - 'http://victoriametrics:8428/api/v1/query_range'

  - username: otel-collector
    password: '${OTEL_PASSWORD}'
    url_prefix:
      - 'http://victoriametrics:8428/api/v1/write'
```

12.2 Compliance checklist for LLM metrics

Compliance point	Recommended implementation
PII in metrics	Never label with user query content. Use opaque IDs only.
Cost audit (GDPR/NIS2)	3-year retention on llm_cost_usd_total. Monthly CSV export.
Encryption in transit	TLS on all VM endpoints in production (vmauth or reverse proxy).
Multi-tenant isolation	vmauth for per-tenant access segmentation. Grafana Teams + RBAC.
Backup	vmbackup incremental to S3-compatible storage (MinIO for air-gap).
Dashboard access	Grafana with SSO (LDAP, OIDC). Viewer-only for business teams.

13. Practical Labs

Lab 1 - Deploy the stack and ingest first metrics

Goal

Deploy the complete Docker Compose stack (VM + OTel Collector + Grafana + VMAAlert).
Ingest metrics from the provided LLM simulator script. Verify in vmui and Grafana.
Estimated time: 30 minutes | Level: beginner

1. Clone the repo and start the stack

```
git clone https://github.com/erythix4/aiobs-lab
cd aiobs-lab/labs/victoriametrics-llm
docker compose up -d
docker compose ps # verify all services are UP
```

2. Run the LLM simulator

```
python simulate_llm_metrics.py --rps 10 --models 3 --duration 300
```

3. Check in vmui (<http://localhost:8428/vmui>)

Query: sum by (model) (rate(llm_requests_total[1m])) * 60

4. Import the Grafana dashboard

Import from: aiobs-lab/labs/victoriametrics-llm/grafana/dashboard-llm-prod.json

Lab 2 - Write 5 MetricsQL queries

Goal

Using metrics generated by the simulator, write the 5 core production queries.
Validate them in vmui, then add them to a Grafana dashboard.
Estimated time: 40 minutes | Level: intermediate

5. Total cost by model over the last 24 hours
6. TTFT P95 by provider (histogram_quantile)
7. RAG retrieval score anomaly (anomaly_score)
8. Top 5 tenants by cost (topk + increase)
9. GPU efficiency: tokens per watt (metric division)

Lab 3 - Configure a production alert

Goal

Configure the RAGIndexDrift alert in VMAAlert. Test it by injecting a score degradation. Verify the notification in the test Slack channel.

Estimated time: 25 minutes | Level: intermediate

10. Copy and adapt alerts/llm-quality.yml with your test Slack webhook
11. Reload VMAAlert: `docker compose restart vmaalert`
12. Inject a score degradation: `python simulate_llm_metrics.py --score-drift 0.5`
13. Verify in VMAAlert UI (<http://localhost:8880>) that the alert fires after 15 min
14. Confirm receipt in Slack

Lab 4 - Recording rules and performance

Goal

Add the 5 essential recording rules. Compare query response times with and without recording rules via the VictoriaMetrics API.

Estimated time: 20 minutes | Level: advanced

Lab 5 - Backup, vmauth and air-gap (bonus, Module 12 hands-on)

Goal Add MinIO (S3-compatible storage), vmbbackup runner and vmauth multi-role reverse proxy. Run a full restore drill from the backup.

Estimated time: 35 minutes | **Level:** advanced

- Start the air-gap overlay:

```
make lab5-airgap
# Or directly:
docker compose -f docker-compose.yml -f airgap/docker-compose.airgap.yml up -d
```

- Verify backups land in MinIO -- open <http://localhost:9001> (admin / admin12345). Bucket vm-backups/auto fills within 5-6 minutes.
- Verify vmauth multi-role authentication:

```
# Read-only role (Grafana)
curl -u grafana:grafana-password 'http://localhost:8427/api/v1/query?query=up'

# Write-only role (OTel collector)
echo 'demo_metric 1' | curl -u otel-collector:otel-password \
  --data-binary @- 'http://localhost:8427/api/v1/write'

# Admin role -- everything
curl -u admin:admin-password 'http://localhost:8427/-/healthy'
```

- Run the disaster-recovery drill (wipe VM data and restore from MinIO). See [solutions/lab5-backup-airgap.md](#) for the exact vmrestore command.

Lab 6 - MetricsQL functions PromQL cannot do (VM-differentiator)

Goal Demonstrate six MetricsQL functions that have no PromQL equivalent. Each one solves a real LLM observability problem.

Estimated time: 30 minutes | **Level:** intermediate

Open vmui (<http://localhost:8428/vmui>) and try the six queries below. Reference solutions in [solutions/lab6-metricsql-exclusive.md](#).

#	Function	What it solves
1	anomaly_score (vmanomaly Enterprise) / manual z-score in OSS	Drift detection without static thresholds
2	outliers_iqr(q)	Light up only the model whose latency just broke

		-- empty when nothing is abnormal
3	share_le_over_time(m[d], threshold)	Direct answer to '% time below SLO'
4	quantiles_over_time("phi", 0.5, 0.95, 0.99, m[d])	P50/P95/P99 from a gauge in ONE query
5	histogram_avg(rate(m_bucket[5m]))	Mean from a histogram, label-mismatch tolerant
6	keep_last_value(m)	Carry last value forward across scrape gaps

Bonus -- range_normalize(...) lets you overlay metrics on different scales on a single chart.

Lab 7 - Cardinality observation and remediation (VM-differentiator)

Goal Use VictoriaMetrics' native /api/v1/status/tsdb endpoint to detect cardinality issues. Inject high cardinality on purpose, observe, then remediate via vmagent relabeling.

Estimated time: 30 minutes | Level: advanced

- Inject 5000 high-cardinality demo series:

```
make lab7-explode    # or: bash scripts/cardinality_explosion.sh
```

- Observe via VM-native endpoints (no exporter needed):

```
# Top 10 labels by cardinality
curl -s 'http://localhost:8428/api/v1/status/tsdb?topN=10' | jq
'.data.seriesCountByLabelName'

# Open vmui Cardinality tab:
# http://localhost:8428/vmui/#/?tab=cardinality
```

- Remediate (3 strategies, see solutions/lab7-cardinality.md):
 - A. labeldrop the offending label at scrape time (trace_id, request_id, session_id)
 - B. Group fine-grained values into a coarse model_family label
 - C. Restrict tenant_id to cost metrics only
- Cleanup: make lab7-cleanup

Lab 8 - vmagent stream aggregation (VM-differentiator)

Goal Pre-roll cost metrics AT INGESTION (not at query time). Reduce cardinality 10x before the data lands in VM. Pattern impossible in plain Prometheus.

Estimated time: 25 minutes | Level: advanced

IMPORTANT flag-name correction -- in vmagent v1.99 the flag is -remoteWrite.streamAggr.config (per remoteWrite), not -streamAggr.config. The latter does not exist and will cause vmagent to crash-loop with 'flag provided but not defined'.

- Enable stream aggregation:

```
make lab8-streamaggr # uncomments the flag in docker-compose.yml and restarts vmagent
```

- After 1 minute (lab interval shortened to 1m for visibility), query the rolled metric:

```
# In vmui:
llm:cost_usd:agg1h
llm:requests:agg1m
llm:tokens:agg1m
```

- Compare cardinality before vs after:

```
curl -s 'http://localhost:8428/api/v1/series/count?match[]=llm_cost_usd_total'
curl -s
'http://localhost:8428/api/v1/series/count?match[]={__name__="llm:cost_usd:agg1h"}'
```

Production retention strategy: keep raw llm_cost_usd_total for 7-day debug, keep llm:cost_usd:agg1h for 36-month cost audit. Same metric, two retentions, impossible in plain Prometheus.

13B. Lab evolution notes (v1.6+)

13B.1 Three ingestion paths run side by side

The lab now demonstrates THREE distinct ways to land the same metric names in VictoriaMetrics, so trainees can compare patterns directly. Row 3 of the VM Differentiators dashboard plots all three P99 latencies next to each other.

Path	Source	Distinguishing label
Path A -- Prometheus client	llm-simulator (prometheus_client) -> vmagent scrape -> VM remote_write	job=llm-simulator
Path B -- OTLP metric push	llm-app (OTel SDK) -> OTLP gRPC -> OTel collector -> VM remote_write	service_name=llm-demo-app, suffix _milliseconds
Path C -- OTLP traces -> spanmetrics	llm-app traces -> spanmetrics CONNECTOR -> RED metrics -> VM	metric duration_milliseconds, dims gen_ai_*, rag_*

13B.2 OTel Collector spanmetrics is now a connector

In OTel Collector >= 0.96, spanmetrics moved from being a processor to a connector. The lab uses the 0.99 image, so the configuration in the guide (spanmetrics under processors:) needs to be moved under connectors: with a separate metrics/spanmetrics pipeline. The corrected snippet:

```
connectors:
  spanmetrics:
    histogram:
      explicit:
        buckets: [50ms, 100ms, 200ms, 500ms, 1s, 2s, 5s, 10s]
    dimensions:
      - name: gen_ai.request.model
      - name: gen_ai.system
      - name: rag.index.id
      - name: rag.pipeline.id
      - name: rag.tenant.id

service:
  pipelines:
    traces:
      receivers: [otlp]
      exporters: [spanmetrics]
    metrics/spanmetrics:
      receivers: [spanmetrics]
      exporters: [prometheusremotewrite]
```

13B.3 Demo loaders -- make every panel show its value

Many VM Differentiators panels are flat in nominal regime, by design (outliers_iqr returns empty, share_le_over_time stays at 0%). The lab ships with demo loader scripts that create exactly the right signal so each row visualizes its feature within 30-60 seconds.

Command	Triggers	Time to visible
make demo-row1	MetricsQL row -- RAG drift + latency outlier on gpt-4o	30 s
make demo-row2	Stream aggregation row -- enables -- remoteWrite.streamAggr.config	60 s
make demo-row3	OTel histograms row -- verifies llm-app + bursts traffic	30 s
make demo-row4	Cardinality row -- pushes 5000 demo series	5 s
make demo-all	All four in sequence (full live POC demo)	60 s
make demo-clean	Reverts everything	30 s

13B.4 Diagnostic and verification commands

Command	What it does
make diagnose	5-step end-to-end health check
make inspect	Tests every panel query and reports [EMPTY]
make diag-vmagent	vmagent status + logs + targets + reachability
make diag-otel	llm-app health + OTel labels actually present + collector logs
make diag-simulator	Which simulator container is actually producing metrics

13B.5 Four Grafana dashboards (Home + 3 audiences)

Grafana now ships with FOUR dashboards (Home + 3 audience-specific). Grafana home page redirects to the Home dashboard via GF_USERS_HOME_DASHBOARD_UID.

Dashboard	UID	Audience
Home -- VictoriaMetrics LLM Lab	lab-home	Everyone (landing page with navigation)
LLM Observability -- Production	llm-prod	Business / LLM Ops (KPIs, SLO, cost)

VM Ops -- Cardinality & Performance	vm-ops	SRE / Platform (vm_* self-metrics)
VM Differentiators -- Demo & Training	vm-diff	POC / sales / trainer (Lab 6/7/8 panels)

13B.6 OTLP exporter histograms in llm-app

The OTLP-instrumented FastAPI app (llm-app, Module 6.1) uses default explicit-bucket histogram aggregation. Exponential histogram aggregation (ExponentialBucketHistogramAggregation) was investigated but is experimental in OTel Python SDK 1.25 and silently fails to flush via PeriodicExportingMetricReader. The View configuration is parked in app/otel_views.py for the day the SDK API stabilizes.

Note also: when the OTel exporter sends a histogram with an instrument unit (e.g. unit="ms"), the prometheusremotewrite exporter appends the unit name to the metric. So llm_request_duration_ms (unit=ms) is exposed in VM as llm_request_duration_ms_milliseconds_bucket. Filter on this name in your dashboards.

13B.7 Companion documents

Document	Audience	Content
TESTING.md	Tester walkthrough	25-min step-by-step from clone to populated dashboards, with expected values
ARCHITECTURE.md	Architect / SRE	15 sections: components, data flows, scaling, security, failure modes (~795 lines)
solutions/lab2-queries.md	Trainee	Reference solutions for the 5 MetricsQL queries
solutions/lab4-recording-rules.md	Trainee	Speed-up benchmarks before/after recording rules
solutions/lab5-backup-airgap.md	Trainee / SRE	vmbackup / vmrestore / vmauth multi-role drill
solutions/lab6-metricsql-exclusive.md	Trainee	6 MetricsQL queries PromQL cannot do, with line-by-line explanation
solutions/lab7-cardinality.md	Trainee / SRE	Cardinality detection + 3 remediation strategies
solutions/lab8-stream-aggregation.md	Trainee / SRE	Stream aggregation production patterns

13B.8 Quick start (one-liner)

```
git clone https://github.com/erythix4/VMLLM.git
```

```
cd VMLLM
make up                # 9-container stack
sleep 60
make diagnose          # 5-step health check
make demo-all         # populate every demo panel

# then open:
#   http://localhost:3000   (admin/admin) -- 4 dashboards
#   http://localhost:8428/vmui -- MetricsQL playground
#   http://localhost:8880   -- vmalert state
#   http://localhost:8001/health -- llm-app OTLP demo
```

14. References and Resources

Official VictoriaMetrics documentation

Resource	URL
VictoriaMetrics Documentation	docs.victoriametrics.com
MetricsQL Reference	docs.victoriametrics.com/metricsql
vmagent	docs.victoriametrics.com/vmagent
vmalert	docs.victoriametrics.com/vmalert
vmauth	docs.victoriametrics.com/vmauth
vmanomaly	docs.victoriametrics.com/anomaly-detection
GitHub VictoriaMetrics	github.com/VictoriaMetrics/VictoriaMetrics

Integrations and tools

Tool	URL / Reference
OTel Collector Contrib	github.com/open-telemetry/opentelemetry-collector-contrib
DCGM Exporter (GPU)	github.com/NVIDIA/dcgm-exporter
vLLM Metrics	docs.vllm.ai/en/latest/serving/metrics.html
LiteLLM Proxy Metrics	docs.litellm.ai/docs/proxy/prometheus
Grafana VM Plugin	grafana.com/grafana/plugins/victoriametrics-metrics-datasource
prometheus-client Python	github.com/prometheus/client_python

Labs and source code

- github.com/erythix4/aiobs-lab : complete VM + OTel + Grafana labs for LLM
- aiobs-lab/labs/victoriametrics-llm : ready-to-use Docker Compose stack
- aiobs-lab/grafana/dashboard-llm-prod.json : importable Grafana dashboard
- aiobs-lab/alerts/ : production VMAAlert rules

Erythix training courses

Training	Duration	Format
VictoriaMetrics: deployment and administration	2 days	Certified VictoriaMetrics

Training	Duration	Format
LLM and RAG observability in production with OTel	2 days	In-house / inter-company
MetricsQL advanced and LLM alerting	1 day	Specialised module
eBPF for advanced observability	3 days	In-house / inter-company

Contact: formations@erythix.io | erythix.io

Erythix | Samuel Desseaux | VictoriaMetrics Training Partner France / Benelux / Germany
Arize AI Training Partner | Skophia | KubeCon Europe 2026